

List of Experiments

1	Chat Program
2	Write a Client server code to display whether the string is palindrome or not
3	Write a Client server code to display sort a set of numbers
4	Write a Client server code to perform File transfer
5	Write a Client server code to display the contents of the requested file
6	Write a Client server code to display Fibonacci series
7	Write a client server code for CRC
8	Write a client server code for hamming code
9	Write a client server code for Bit Stuffing
10	Write a client server code for Byte Stuffing
11	Write a client server code for Selective Repeat
12	Write a client server code for Go back N
13	Write a client server code for leaky bucket algorithm
14	Write a client server code for Token bucket algorithm
15	Write a client server code that 5 clients will send one number to the server. Server has to sort all numbers and send the final sorted output to all clients. Or client may send more than one numbers

Points to be remembered

- TCP is connection-oriented, UDP is connectionless.
- `socket.AF_INET` → IPv4
- `socket.SOCK_STREAM` → TCP
- `socket.SOCK_DGRAM` → UDP
- `bind()` → assigns IP and port
- `listen()` → waits for connections
- `accept()` → accepts client
- `connect()` → client connects to server
- `send()` / `recv()` → data transmission

Concept

- Server waits for client connection.
- Client sends message.
- Server responds.

Important Socket Functions

Function	Purpose
<code>socket()</code>	Create socket
<code>bind()</code>	Attach IP & port (server)
<code>listen()</code>	Wait for client
<code>accept()</code>	Accept connection
<code>connect()</code>	Connect to server
<code>send()</code>	Send data
<code>recv()</code>	Receive data
<code>close()</code>	Close socket

Simple TCP Client-Server Program (Chat-Program)

Server Code	Meaning									
import socket	Imports Python's socket library for network communication.									
# Create socket server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)	<table><tr><th>Part</th><th>Meaning</th><th>Value</th></tr><tr><td>AF_INET</td><td>IPv4 addressing</td><td>127.0.0.1</td></tr><tr><td>SOCK_STREAM</td><td>TCP protocol</td><td>Reliable connection</td></tr></table> Creates a TCP socket object.	Part	Meaning	Value	AF_INET	IPv4 addressing	127.0.0.1	SOCK_STREAM	TCP protocol	Reliable connection
Part	Meaning	Value								
AF_INET	IPv4 addressing	127.0.0.1								
SOCK_STREAM	TCP protocol	Reliable connection								
# Bind to IP and Port server_socket.bind(("localhost", 12345))	Binds server to: IP address = localhost → 127.0.0.1 Port number = 12345 So server listens on: 127.0.0.1 : 12345									
# Listen for client server_socket.listen(1)	Server can handle 1 client at a time.									
print("Server is waiting for connection...") conn, addr = server_socket.accept() print("Connected with:", addr)	Pauses execution until client connects. Example value after client connects: conn = <socket object> addr = ('127.0.0.1', 53421) # client's IP and random port									
data = conn.recv(1024).decode() print("Client says:", data)	Receives up to 1024 bytes. Example: Client sends: "Hello from Client" data = "Hello from Client"									
conn.send("Hello from Server".encode())	Server sends back: "Hello from Server"									
conn.close() server_socket.close()	Closes the connection and the server.									
Client Code	Meaning									
import socket	Imports socket module.									
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)	Creates TCP client socket.									
client_socket.connect(("localhost", 12345))	Client connects to: Server IP = 127.0.0.1 Port = 12345									
client_socket.send("Hello from Client".encode())	Client sends: "Hello from Client"									
data = client_socket.recv(1024).decode() print("Server says:", data)	Client receives: "Hello from Server"									

client_socket.close()	Closes client socket.
-----------------------	-----------------------

Key Points

- Port 12345 is user-defined.
- recv(1024) means receive max 1024 bytes.
- encode() converts string → bytes.
- decode() converts bytes → string.
- Client uses random port (like 53421).
- Server uses fixed port.

Runtime Flow with Values

Step	Action	Data
1	Server binds	127.0.0.1:12345
2	Server waits	listen()
3	Client connects	connect()
4	Server accepts	addr = ('127.0.0.1', 53421)
5	Client sends	"Hello from Client"
6	Server receives	"Hello from Client"
7	Server replies	"Hello from Server"
8	Client receives	"Hello from Server"

Write a Client server code to display whether the string is palindrome or not

Server Code	Client Code
<pre>import socket def is_palindrome(data): data = data.lower() data = data.replace(" ", "") n = len(data) for i in range(n // 2): if data[i] != data[n - i - 1]: return "Not a Palindrome" return "Palindrome" server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server_socket.bind(("localhost", 12345)) server_socket.listen(5) print("Palindrome Server running at 127.0.0.1:12345") while True: print("\nWaiting for client...") conn, addr = server_socket.accept() print("Connected to:", addr) data = conn.recv(1024).decode() print("Received:", data) result = is_palindrome(data) conn.send(result.encode()) print("Sent result:", result) conn.close()</pre>	<pre>import socket client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) client_socket.connect(("localhost", 12345)) msg = input("Enter a string: ") client_socket.send(msg.encode()) data = client_socket.recv(1024).decode() print("Server Response:", data) client_socket.close()</pre> How to Run Step 1 — Start Server python pal_server.py Step 2 — Run Client (new terminal) python pal_client.py Sample Output Client: Enter a string: radar Server Response: Palindrome Server: Waiting for client... Connected to: ('127.0.0.1', 54021) Received: radar Sent result: Palindrome

Write a Client server code to display sort a set of numbers

Server Code map()-> Converts each string number into an integer.	Client Code
<pre>import socket def bubble_sort(nums): n = len(nums) for i in range(n): for j in range(0, n - i - 1): if nums[j] > nums[j + 1]: nums[j], nums[j + 1] = nums[j + 1], nums[j] return nums server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server_socket.bind(("localhost", 12346)) server_socket.listen(5) print("Sorting Server running at 127.0.0.1:12346") while True: print("\nWaiting for client...") conn, addr = server_socket.accept() print("Connected to:", addr) data = conn.recv(1024).decode() print("Received:", data) nums = list(map(int, data.split(","))) sorted_nums = bubble_sort(nums) result = ",".join(map(str, sorted_nums)) conn.send(result.encode()) print("Sent sorted data:", result) conn.close()</pre>	<pre>import socket client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) client_socket.connect(("localhost", 12346)) msg = input("Enter numbers separated by commas: ") client_socket.send(msg.encode()) data = client_socket.recv(1024).decode() print("Sorted Numbers:", data) client_socket.close()</pre> How to Run Step 1 — Start Server python sort_server.py Step 2 — Run Client (new terminal) python sort_client.py Sample Execution Client: Enter numbers separated by commas: 5,2,9,1,3 Sorted Numbers: 1,2,3,5,9 Server: Connected to: ('127.0.0.1', 54210) Received: 5,2,9,1,3 Sent sorted data: 1,2,3,5,9

5 clients will send one number to the server. Server has to sort all numbers and send the final sorted output to all clients. Or client may send more than one numbers. Replace yellow color code with box contents

How It Works

1. Server accepts 5 clients.
2. Server stores all numbers in a list.
3. Server manually sorts using **Bubble Sort**.
4. Server sends sorted result back to all clients.

Server Code	Client code
<pre>import socket HOST = '127.0.0.1' PORT = 5000 def bubble_sort(arr): n = len(arr) i = 0 while i < n: j = 0 while j < n - i - 1: if arr[j] > arr[j + 1]: temp = arr[j] arr[j] = arr[j + 1] arr[j + 1] = temp j = j + 1 i = i + 1 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server.bind((HOST, PORT)) server.listen(5)</pre>	<pre>import socket HOST = '127.0.0.1' PORT = 5000 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) client.connect((HOST, PORT)) num = input("Enter one number: ") client.send(num.encode()) result = client.recv(1024).decode() print("From server:", result) client.close() nums = input("Enter numbers separated by comma: ") client.send(nums.encode())</pre> 

```
print("Server waiting for 5 clients...")
```

```
numbers = []  
clients = []
```

```
while len(numbers) < 5:  
    conn, addr = server.accept()  
    print("Connected from", addr)
```

```
data = conn.recv(1024).decode()  
num = int(data)  
numbers.append(num)
```

```
clients.append(conn)
```

```
print("Received:", num)  
add this statement
```

```
# Manual sort  
bubble_sort(numbers)
```

```
result = "Sorted numbers: "  
i = 0  
while i < len(numbers):  
    result = result + str(numbers[i]) + " "  
    i = i + 1
```

```
print(result)
```

```
# Send result to all clients  
for c in clients:  
    c.send(result.encode())  
    c.close()
```

```
server.close()
```

```
count = 0  
while count < 5:
```

```
data = conn.recv(1024).decode()  
parts = data.split(',')  
  
i = 0  
while i < len(parts):  
    numbers.append(int(parts[i]))  
    i = i + 1
```

```
count = count + 1
```

Write a Client server code to perform File transfer

Server Code	Client Code
<pre>import socket s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) print("Waiting...") c, addr = s.accept() print("Connected:", addr) f = open("received.txt", "wb") data = c.recv(1024) while data: f.write(data) data = c.recv(1024) f.close() c.close() s.close() print("File received")</pre>	<pre>import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) f = open("send.txt", "rb") data = f.read(1024) while data: s.send(data) data = f.read(1024) f.close() s.close() print("File sent")</pre>
How to Run 1. Create a file named send.txt in the same folder as client.py. 2. Open two command prompts. Terminal 1 (Server): python server.py Terminal 2 (Client): python client.py	rb = Read Binary (read file as raw bytes) wb = Write Binary (write file as raw bytes) Result send.txt → transmitted → saved as received.txt in server folder.

Write a Client server code to display the contents of the requested file

Server	Client
<pre>import socket import os s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) c, a = s.accept() name = c.recv(1024).decode() if os.path.exists(name): f = open(name, "r") data = f.read() f.close() c.send(data.encode()) else: c.send("File not found".encode()) c.close() s.close()</pre>	<pre>import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) name = input("Enter file name: ") s.send(name.encode()) data = s.recv(4096).decode() print(data) s.close()</pre>
How to Run 1. Put a text file (e.g., test.txt) in server folder. 2. Run server: 3. python server.py 4. Run client: 5. python client.py Enter: test.txt	Result Client prints the contents of test.txt.

Write a Client server code to display Fibonacci series

Server Code	Client code
<pre> import socket s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) c, a = s.accept() n = int(c.recv(1024).decode()) x = 0 y = 1 res = "" for i in range(n): res = res + str(x) + " " temp = x + y x = y y = temp c.send(res.encode()) c.close() s.close() </pre>	<pre> import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) n = input("Enter the value of n") s.send(n.encode()) print(s.recv(4096).decode()) s.close() </pre>
Why str(x)? Because: • x is a number (int) • res is text (string) • Python cannot directly add a number to a string So we convert x into a string: str(x)	What does " " mean? That is a space character. It is added so that the numbers do not stick together. Without space: 011235 With space: 0 1 1 2 3 5

Write a client server code for CRC

Server Code	Meaning	Client Code
<pre>import socket s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) c, a = s.accept() data = c.recv(1024).decode() div = c.recv(1024).decode()</pre>		<pre>import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) data = input("Enter the data:") div = input("Enter the divisor") s.send(data.encode()) s.send(div.encode()) rem = s.recv(1024).decode() print(rem) s.close()</pre>
data = data + "0" * (len(div) - 1)	Adds extra zeros to prepare for CRC division.	Run Start server: python server.py Start client: python client.py Enter: Enter the data:10101 Enter the divisor :1101 Output: 001
data = list(data) div = list(div)	Convert strings to lists which allows modifying individual bits.	
for i in range(len(data) - len(div) + 1): if data[i] == '1': for j in range(len(div)): if data[i + j] == div[j]: data[i + j] = '0' else: data[i + j] = '1'	Performs bit-by-bit XOR division to compute remainder.	
rem = "" for i in range(len(div) - 1): rem = rem + data[-(len(div) - 1) + i]	Extract remainder. Collects the last len(div) - 1 bits as CRC remainder.	
c.send(rem.encode())	Send remainder to client. Sends computed CRC remainder.	

c.close() s.close()	Close connections. Closes client and server sockets.	
--------------------------------------	--	--

Write a client server code for hamming code

Server Code	Client code
<pre>import socket s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) c, a = s.accept() data = c.recv(1024).decode() m = len(data) # find number of parity bits r = 0 while (2**r) < (m + r + 1): r += 1 #Total bits = data bits + parity bits. n = m + r # Create array filled with 0s. code = ['0'] * (n + 1) # place data bits #Positions 1,2,4,8,16 are parity bits → skip them #Fill remaining positions with data bits j = 0 for i in range(1, n + 1): if i == 1 or i == 2 or i == 4 or i == 8 or i == 16: continue else: code[i] = data[j] j += 1 # calculate parity bits for i in range(r): pos = 2**i parity = 0 for j in range(1, n + 1): if j & pos: parity ^= int(code[j]) code[pos] = str(parity)</pre>	<pre>import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) d = input("Enter binary data: ") s.send(d.encode()) print("Hamming Code:", s.recv(1024).decode()) s.close()</pre>

#Join bits from index 1 to n → final Hamming code.

```
result = ".join(code[1:])
```

```
c.send(result.encode())
```

```
c.close()
```

```
s.close()
```

How It Works:

Input: 4 bits

d1 d2 d3 d4

Parity bit calculation:

Parity	Covers bits	Formula
p1	d1, d2, d4	$p1 = d1 \oplus d2 \oplus d4$
p2	d1, d3, d4	$p2 = d1 \oplus d3 \oplus d4$
p3	d2, d3, d4	$p3 = d2 \oplus d3 \oplus d4$

Bit positions in final 7-bit code:

Position	Bit
1	p1
2	p2
3	d1
4	p3
5	d2
6	d3
7	d4

Example

Input:

1011

Calculation:

$d1=1, d2=0, d3=1, d4=1$

$p1 = 1 \oplus 0 \oplus 1 = 0$

$p2 = 1 \oplus 1 \oplus 1 = 1$

$p3 = 0 \oplus 1 \oplus 1 = 0$

Output:

0110011

Write a client server code for bit stuffing

Server	Client
<pre>import socket s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) c, a = s.accept() data = c.recv(1024).decode() count = 0 res = "" for bit in data: if bit == '1': count = count + 1 res = res + '1' if count == 5: res = res + '0' count = 0 else: res = res + '0' count = 0 c.send(res.encode()) c.close() s.close()</pre>	<pre>import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) data = input("Enter bit stream: ") s.send(data.encode()) stuffed = s.recv(1024).decode() print("Stuffed bits:", stuffed) s.close()</pre>

How It Works (Simple Explanation)

Rule:

After five consecutive 1s, insert a 0.

Example:

Input:

1111101111110

Processing:

11111 → insert 0 → 111110

11111 → insert 0 → 111110

Output:

11111001111100

Write a client server code for byte stuffing

Server Code	Client Code
<pre>import socket s = socket.socket() s.bind(("127.0.0.1", 9999)) s.listen(1) c, a = s.accept() data = c.recv(1024).decode() res = "" for ch in data: if ch == 'F' or ch == 'E': res = res + 'E' res = res + ch c.send(res.encode()) c.close() s.close()</pre>	<pre>import socket s = socket.socket() s.connect(("127.0.0.1", 9999)) data = input("Enter data: ") s.send(data.encode()) stuffed = s.recv(1024).decode() print("Stuffed data:", stuffed) s.close()</pre>
How It Works (Simple Explanation) Rule: If data contains F or E, prefix it with E.	Example Input: ABFCD Processing: F → EF Output: ABEFCD Another example: Input: AEEF Output: AEEEEF

Write a client server code for selective repeat protocol

Server Code

```
import socket                # Import socket module for networking
import random                # Import random module for packet loss simulation

HOST = '127.0.0.1'          # Server IP address
PORT = 5000                 # Server port number

TOTAL = 5                   # Total number of frames
LOSS = 0.3                  # 30% probability of packet loss

received = [False] * TOTAL  # List to track received frames

s = socket.socket()         # Create a socket object
s.bind((HOST, PORT))        # Bind socket to IP and port
s.listen(1)                 # Listen for incoming connection

print("Server ready...")    # Print server status
conn, addr = s.accept()     # Accept client connection
print("Connected from", addr) # Print client address
```


Client Code

```
import socket          # Import socket module
import time            # Import time module for timeout tracking
import select          # Import select module for non-blocking receive

HOST = '127.0.0.1'     # Server IP address
PORT = 5000            # Server port number
                        Ctrl+N

TOTAL = 5              # Total number of frames
WINDOW = 3             # Sliding window size
TIMEOUT = 2            # Timeout duration in seconds

acked = [False] * TOTAL # List to track acknowledgements
send_time = {}          # Dictionary to store frame send times

base = 0               # Base of the sliding window
next_seq = 0           # Next frame number to send

s = socket.socket()    # Create socket
s.connect((HOST, PORT)) # Connect to server
```

[illegible]

Explanation

Step	What happens
Client sends frames	Multiple frames sent in a window
Server randomly drops some frames	Simulates network loss
Server sends ACK for received frames	Confirms delivery
Client waits for ACK	If not received, timeout occurs
Client resends only that frame	Selective Repeat

Write a client server code for Go-Back-N ARQ protocol: This program implements Go-Back-N ARQ by sending frames in a sliding window and retransmitting all frames from the lost frame onward when a timeout occurs.

Server Code

```
import socket                # Import socket module for networking
import random                # Import random module for packet loss simulation

HOST = '127.0.0.1'          # Server IP address
PORT = 5000                 # Server port number

TOTAL = 5                   # Total number of frames
LOSS = 0.3                  # 30% probability of packet loss

expected = 0                # Next expected frame number

s = socket.socket()         # Create a socket object
s.bind((HOST, PORT))        # Bind socket to IP and port
s.listen(1)                 # Listen for incoming connections

print("Server ready...")    # Print server status
conn, addr = s.accept()     # Accept client connection
print("Connected from", addr) # Print client address
```

```
while expected < TOTAL:           # Loop until all frames received

    data = conn.recv(1024).decode() # Receive data from client
    if not data:                    # If no data received
        break                       # Exit Loop

    frame = int(data)               # Convert received data to integer

    if random.random() < LOSS:      # Randomly simulate packet loss
        print("Frame lost:", frame) # Print loss information
        continue                   # Skip processing

    if frame == expected:           # If this is the expected frame
        print("Received frame:", frame)
        ack = "ACK" + str(frame)   # Create ACK
        conn.send(ack.encode())    # Send ACK to client
        print("Sent", ack)
        expected += 1              # Move to next expected frame
    else:
        print("Discarded frame:", frame)
        ack = "ACK" + str(expected - 1) # Send last correct ACK
        conn.send(ack.encode())
        print("Resent", ack)
```

```
print("All frames received.")
conn.close()
s.close()
```

Client Code

```
import socket                # Import socket module
import time                  # Import time module for timeout tracking
import select                # Import select for non-blocking receive

HOST = '127.0.0.1'          # Server IP address
PORT = 5000                  # Server port number
Ctrl+N

TOTAL = 5                    # Total number of frames
WINDOW = 3                   # Window size
TIMEOUT = 2                  # Timeout duration

base = 0                     # Base of sliding window
next_seq = 0                 # Next frame to send

send_time = {}               # Dictionary to store send times

s = socket.socket()          # Create socket
s.connect((HOST, PORT))      # Connect to server
```

```

while base < TOTAL:                                # While frames remain

    while next_seq < base + WINDOW and next_seq < TOTAL: # Send frames within window
        print("Sending frame:", next_seq)
        s.send(str(next_seq).encode())
        send_time[next_seq] = time.time()
        next_seq += 1

    ready, _, _ = select.select([s], [], [], 1) # Check if ACK available

    if ready:
        data = s.recv(1024).decode()
        if data.startswith("ACK"):
            ack = int(data[3:])
            print("Received", data)

            if ack >= base:
                base = ack + 1

    if base < next_seq and time.time() - send_time[base] > TIMEOUT:
        print("Timeout! Resending from frame:", base)

        for i in range(base, next_seq):
            print("Resending frame:", i)
            s.send(str(i).encode())
            send_time[i] = time.time()

```

```
print("All frames sent.")  
s.close()
```

Key Difference between GBN and SR

Feature	Selective Repeat	Go-Back-N
ACK type	Individual ACK	Cumulative ACK
Retransmission	Only lost frames	All frames after lost
Receiver	Accepts out-of-order	Discards out-of-order